MLOPS ASSIGNMENT 1

Heart Disease Risk Prediction

An end-to-end MLOps pipeline: experiment tracking, CI/CD, containers, Kubernetes and monitoring

Name: DESHPANDE ASHUTOSH ABHIJIT PRITEE

Student ID: 2025AF05178

Date: 11 October 2026

Code repository: https://github.com/ashutoshdeshpande91/MLOps_Assign1

1. Introduction

This project builds a classifier for heart diseases. They are leading cause of death, and early risk assessment from routine clinical measurements can help prioritise patients for further tests. It predicts whether a patient has heart disease from 13 health measurements and deploys it as a monitored web service.

The aim is to create a reproducible process rather than only an accurate model. The project therefore covers the full lifecycle including:

- data download, cleaning and exploratory analysis (EDA), all as re-runnable scripts;
- model training and tuning with every experiment tracked in MLflow;
- automated linting, unit tests and training on every push, using GitHub Actions;
- a Docker container serving the model through a FastAPI `/predict` endpoint;
- deployment to a local Kubernetes cluster (Minikube) behind a LoadBalancer service;
- request logging and Prometheus and Grafana monitoring.

All code, tests, configuration and screenshots are in the GitHub repository shared on the first page. Figure 1 in Section 6 shows how all the pieces fit together.

2. Setup and installation

The project needs Python 3.13 and Git. Docker is needed for the container, Kubernetes and monitoring parts.

2.1 Install

```
git clone https://github.com/ashutoshdeshpande91/MLOps_Assign1.git
cd MLOps_Assign1
python -m venv .venv
source .venv/Scripts/activate     # Git Bash on Windows
pip install -r requirements.txt -r requirements-api.txt -r requirements-dev.txt
```

All dependency files pin exact versions, so a fresh environment reproduces the one used for the results in this report.

2.2 Run the pipeline

```
python src/download_data.py     # raw data to data/raw/
python src/data_processing.py   # cleaned data to data/heart_clean.csv
python src/eda.py               # EDA plots to screenshots/eda/
python src/train.py             # train, tune, track in MLflow, save the model
python src/predict.py           # prediction for a sample patient
flake8 src tests && pytest      # lint and unit tests
```

The cleaned dataset is committed to the repository, so training also runs without downloading anything. The `README.md` in the repository gives the commands for Docker, Kubernetes and monitoring.

3. Data and exploratory analysis

3.1 Dataset

The data is the Cleveland subset of the UCI Heart Disease dataset: 303 patients, 13 clinical features and a diagnosis column. I have split the features by type, because the type decides how each one is pre-processed.

Feature	Meaning	Type
age	Age in years	numeric
trestbps	Resting blood pressure (mm Hg)	numeric
chol	Serum cholesterol (mg/dl)	numeric
thalach	Maximum heart rate achieved	numeric
oldpeak	ST depression induced by exercise	numeric
sex	1 = male, 0 = female	categorical
cp	Chest pain type (1 to 4)	categorical
fbs	Fasting blood sugar above 120 mg/dl	categorical
restecg	Resting ECG result (0 to 2)	categorical
exang	Exercise-induced angina	categorical
slope	Slope of the peak exercise ST segment	categorical
ca	Major vessels coloured by fluoroscopy (0 to 3)	categorical
thal	Thallium test result (3 normal, 6 fixed defect, 7 reversible defect)	categorical

3.2 Cleaning

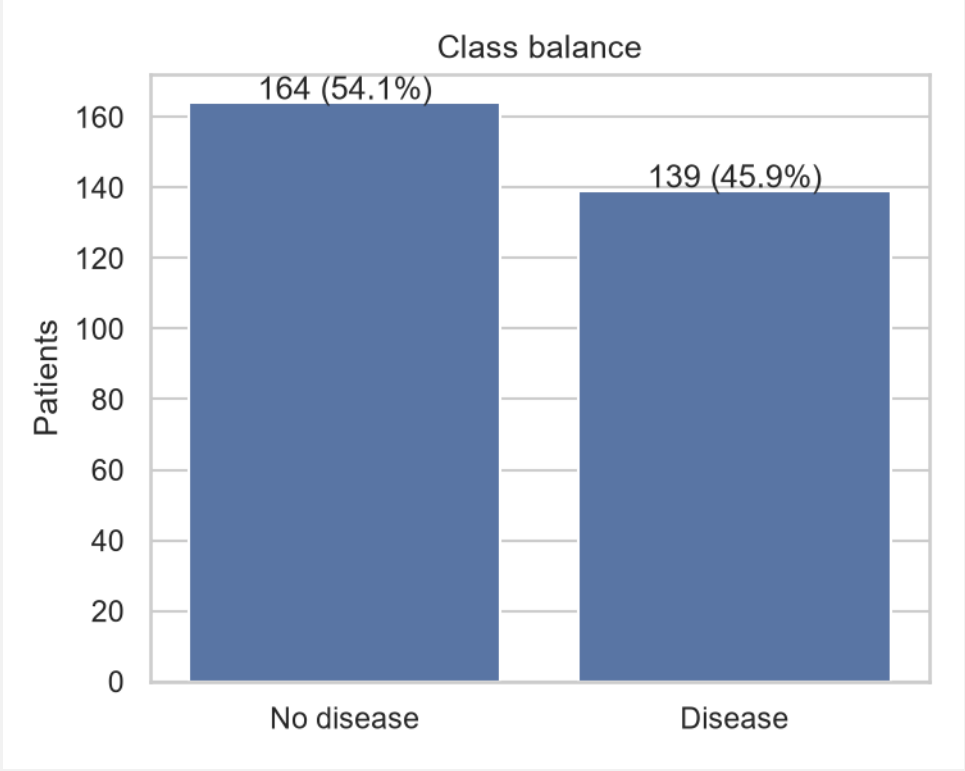
- Missing values are stored as `?` in the source file. Only ca (4 patients) and thal (2 patients) have any. I filled numeric gaps with the median and categorical gaps with the most frequent value, which keeps all 303 patients instead of discarding rows from a small dataset.
- Duplicate rows are dropped (the function is there as a safeguard).
- The original diagnosis takes the values 0 to 4 (0 = no disease, 1 to 4 = increasing severity). I converted it to a binary target: 0 = no disease, 1 = disease present.
- `ca` and `thal` look numeric but are coded values, so I have treated them as categorical along with the other coded columns.

Scaling and one-hot encoding are not done during cleaning. They live inside the model pipeline (Section 4), so they are fitted only on training data and are applied identically when the model serves requests.

3.3 Exploratory analysis

`src/eda.py` produces six plots in `screenshots/eda/`: class balance, histograms and boxplots of the numeric features split by diagnosis, count plots of the categorical features split by diagnosis, a correlation heatmap and a ranking of features by correlation with the target.

The classes are close to balanced: about 54% of patients have no disease and 46% have disease Because of this I did not use resampling or class weights, and I have reported accuracy alongside precision, recall and ROC-AUC



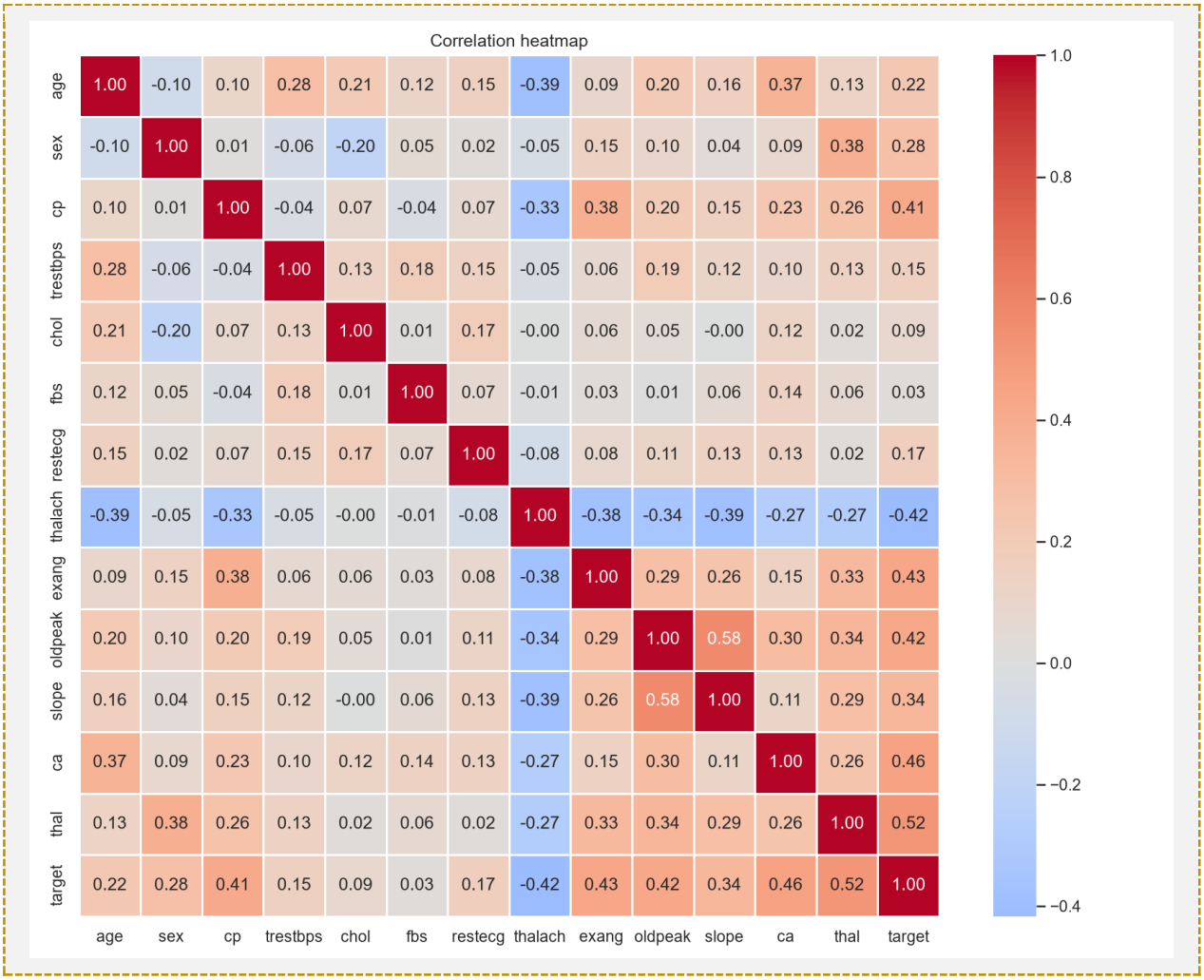


Figure 2: Class balance and feature correlations.

4. Feature engineering and modelling

4.1 Preprocessing pipeline

A scikit-learn `ColumnTransformer` standardises the 5 numeric features (zero mean, unit variance) and one-hot encodes the 8 categorical features, ignoring unseen categories at prediction time. It is combined with the classifier in a single `Pipeline`. This has two benefits: during cross-validation the scaler and encoder are fitted only on each training fold, which avoids data leakage, and the saved model accepts raw patient values, so the API needs no separate preprocessing code.

4.2 Models, split and tuning

I compared two models that behave differently. Logistic Regression is an interpretable linear baseline. Random Forest can capture non-linear effects and feature interactions.

Setting	Value
Train / test split	80% / 20%, stratified (242 / 61 patients), random_state 42
Cross-validation	5-fold stratified, shuffled, scored on ROC-AUC
Logistic Regression grid	C in {0.01, 0.1, 1, 10} (4 candidates), max_iter 1000

Setting	Value
Random Forest grid	n_estimators in {100, 200}, max_depth in {3, 5, None}, min_samples_split in {2, 5} (12 candidates)
Best Logistic Regression	C = 1
Best Random Forest	n_estimators = 100, max_depth = None, min_samples_split = 5

I used ROC-AUC to pick hyperparameters because it measures how well the model ranks patients by risk regardless of the decision threshold, and it is stable on a slightly imbalanced set.

4.3 Results

Model	CV acc.	CV prec.	CV recall	CV ROC-AUC	Test acc.	Test prec.	Test recall	Test ROC-AUC
Logistic Regression	0.847	0.877	0.783	0.902	0.885	0.839	0.929	0.966
Random Forest	0.814	0.799	0.801	0.898	0.885	0.839	0.929	0.955

Table: cross-validated metrics (mean over 5 folds on the training split) and held-out test metrics.

4.4 Model selection

I selected **Logistic Regression**. It has the higher cross-validated ROC-AUC (0.902 against 0.898), higher cross-validated accuracy (0.847 against 0.814) and a smaller spread between folds (ROC-AUC standard deviation 0.014 against 0.023). The ROC-AUC gap is small, well inside one standard deviation, so the two models are close in quality. With no clear accuracy advantage for the forest, the simpler, more stable and more interpretable model is the better choice for a clinical setting.

The test metrics need careful reading. Both models make the same predictions on the 61 test patients: 54 correct, with an implied 26 true positives, 28 true negatives, 5 false positives and 2 false negatives (derived from the reported accuracy, precision and recall). Each patient is worth about 1.6 percentage points, so small differences on this test set mean little, and the cross-validated numbers are the more reliable guide. The test ROC-AUC (0.966) is also higher than the cross-validated one (0.902), which most likely reflects a favourable split rather than real improvement.

Recall is the metric that matters most for screening, because a missed patient is the costly error. Cross-validated recall (0.78) is lower than test recall (0.93), which shows how much it varies on a dataset this small. If recall had to be raised, the decision threshold could be lowered below 0.5 at the cost of more false positives.

5. Experiment tracking

`src/train.py` logs every run to MLflow, stored locally in `mlflow.db` and `mlruns/` (SQLite backend), under the experiment `heart-disease-classification`. One training session creates 18 runs:

- **2 main runs**, one per model (`logistic_regression`, `random_forest`);
- **16 child runs**, one per hyperparameter combination tried (4 for Logistic Regression, 12 for Random Forest), each with its parameters and cross-validated ROC-AUC, so the tuning process can be inspected and compared.

Logged on each main run	Details
Parameters	model type, CV folds, test size, random state, train and test sizes, feature lists, best hyperparameters
Metrics	cross-validated accuracy, precision, recall and ROC-AUC (with standard deviations), and the four test metrics
Plots	ROC curve, confusion matrix, feature importance or coefficients
Artifacts	the cleaned dataset snapshot and the fitted pipeline with its input signature
Tags	`run_type` (best_model or grid_search_candidate); `selected_for_serving` on the chosen model

The selected model is also saved outside MLflow as `models/heart_model.joblib` with a `model_metadata.json` file recording its metrics, input features, library versions and the MLflow run ID, which links the served model back to its experiment.

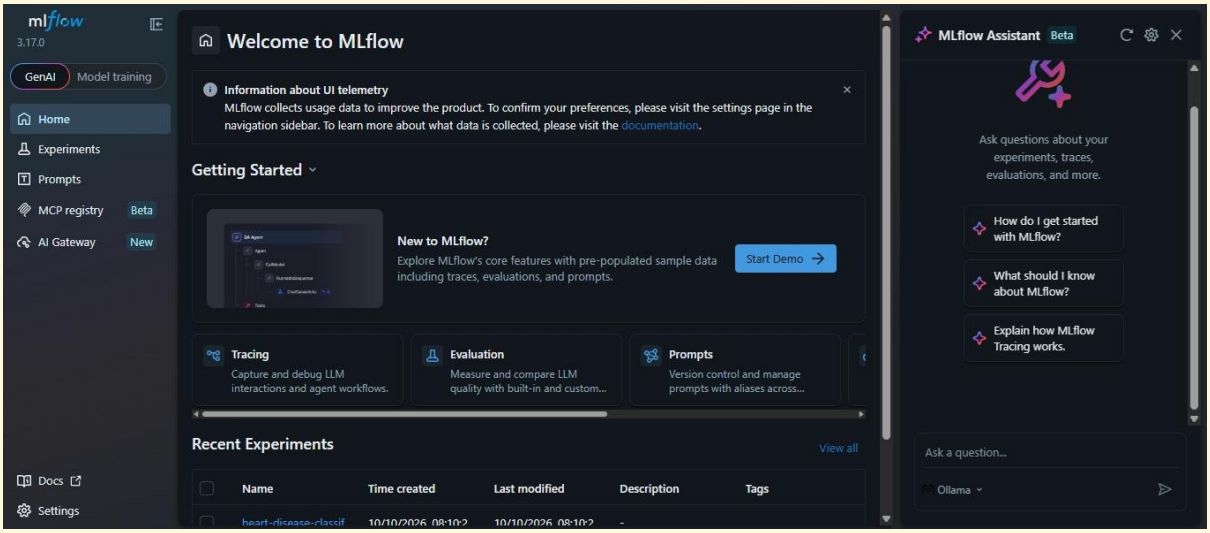
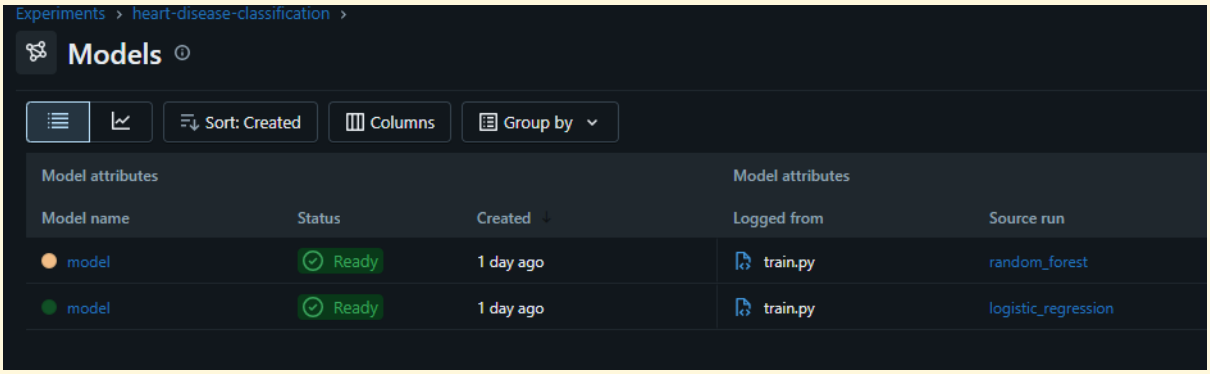


Figure 3: MLflow experiment view with the main and child runs.



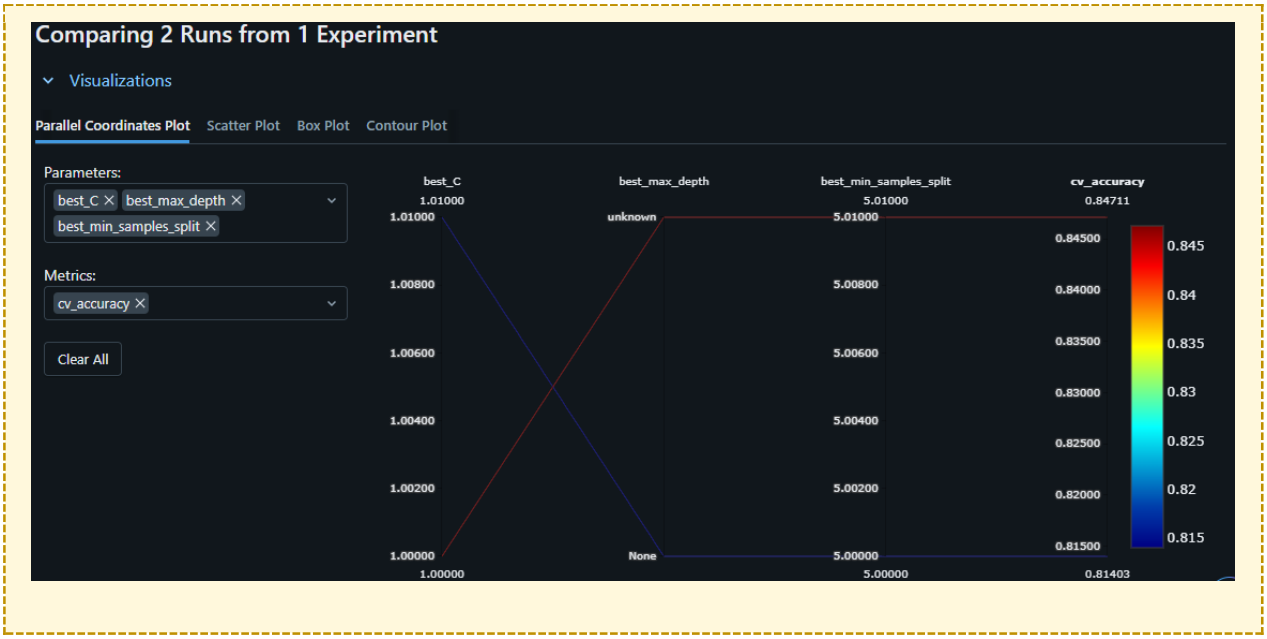


Figure 4: Comparison of the two models in MLflow.

6. System architecture

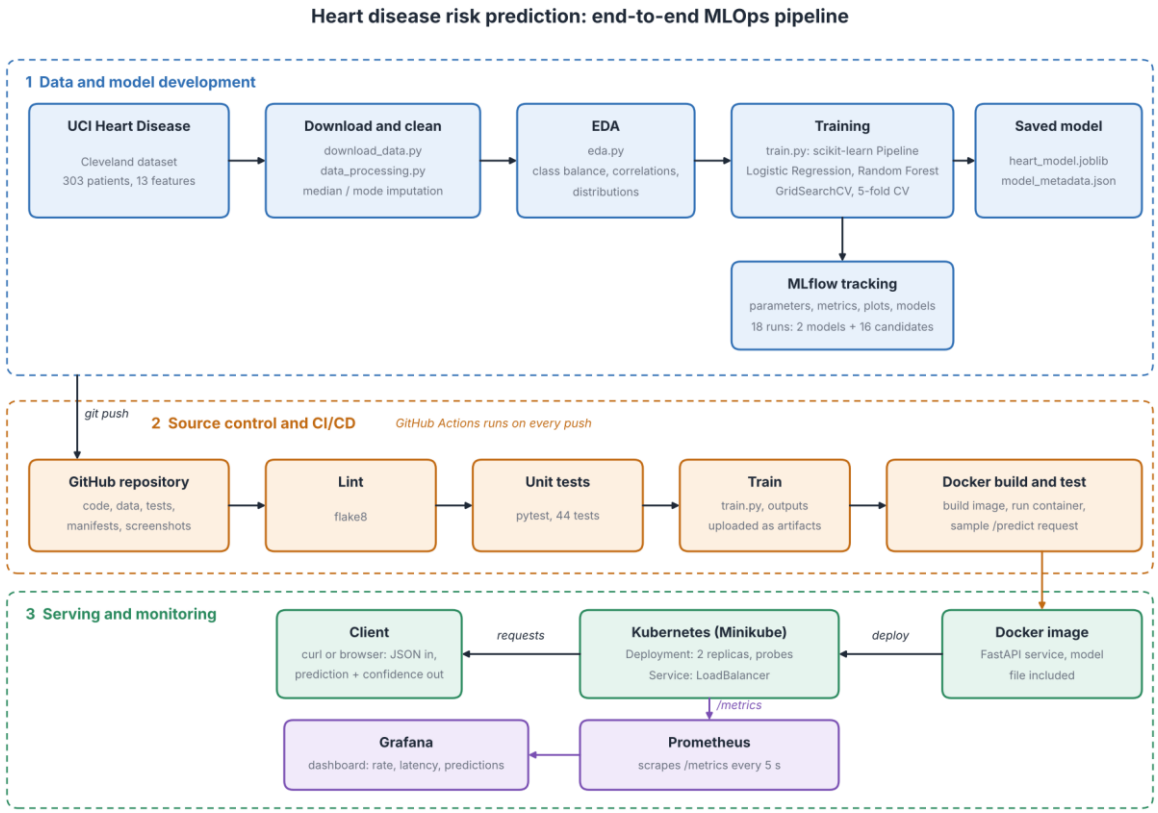


Figure 1: Architecture of the pipeline.

The pipeline has three stages. In the **development** stage, scripts download and clean the data, produce the EDA plots, and train and track the models. In the **CI/CD** stage, a push to GitHub triggers lint, tests, training and a Docker build with a live test of the container. In the **serving** stage, the

image runs on Kubernetes behind a LoadBalancer service, and Prometheus scrapes the API's `/metrics` endpoint while Grafana displays the results.

7. CI/CD pipeline

The workflow in `.github/workflows/ci.yml` runs on every push and pull request to `main`, and can also be started by hand from the Actions tab. It has three jobs:

1. **Lint and unit tests.** Installs the pinned dependencies, runs `flake8` on `src` and `tests`, then `pytest` (44 tests). The JUnit report is uploaded as an artifact even when tests fail.
2. **Train model.** Runs `src/train.py` and a prediction smoke test. The training log, results, model and MLflow data are uploaded as artifacts.
3. **Build and test Docker image.** Builds the image, starts the container, waits for `/health` to respond, then sends a real `/predict` request and prints the container logs.

Jobs 2 and 3 declare `needs: lint-and-test`, so they run only when linting and tests pass. A lint error or failing test stops the pipeline, and the log of the failing step shows the cause.

7.1 Unit tests

The 44 tests use small synthetic data, so they need no network access or downloaded dataset and run in a couple of seconds. They cover:

- **Data processing:** missing-value handling (median and mode), target binarisation, duplicate removal, no mutation of the input, correct column types, reading the raw file.
- **Model code:** preprocessing output, unseen categories, valid probabilities, reproducibility with a fixed seed, evaluation metrics, saving and reloading the model.
- **Inference and API:** response fields, missing-feature errors, input validation (HTTP 422), the 503 response when the model is absent, request-ID headers, request logging and the metrics endpoint.

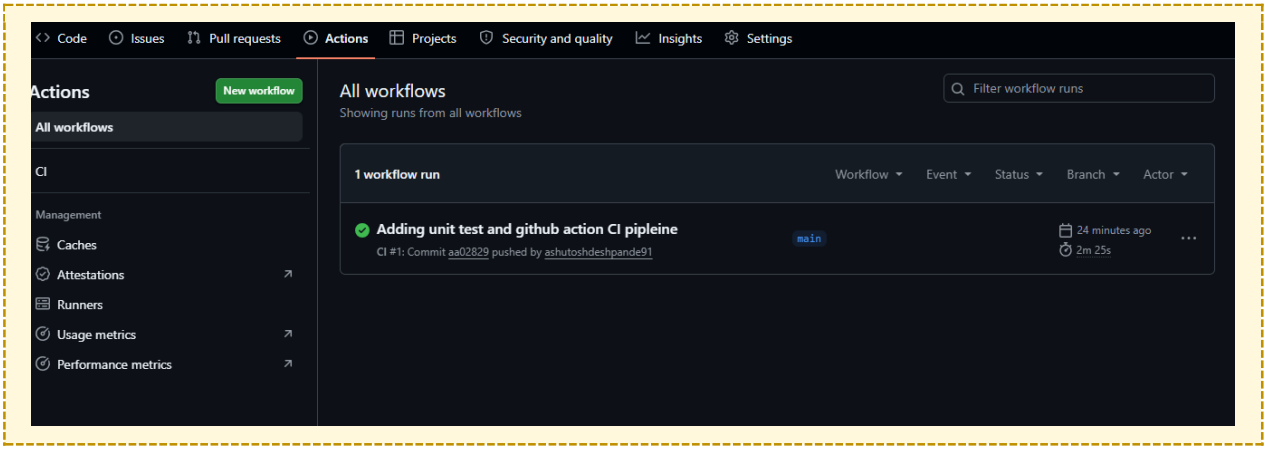


Figure 5: A successful pipeline run.

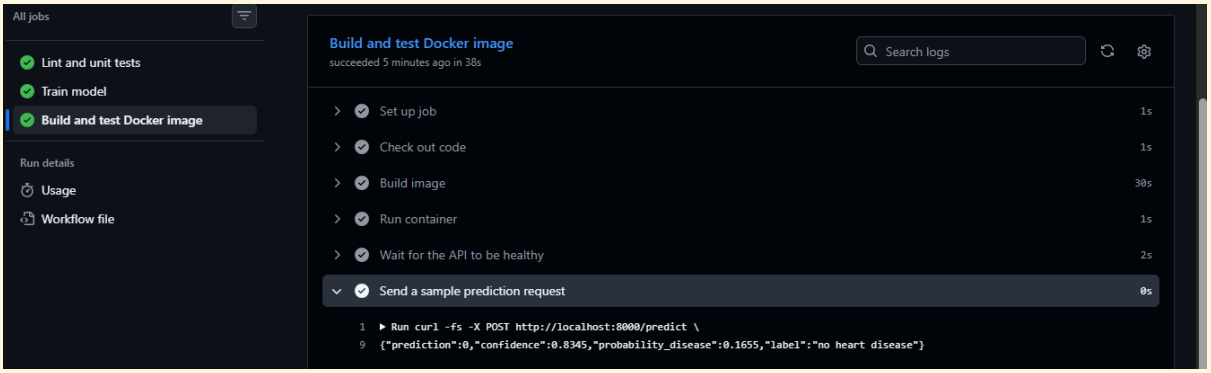
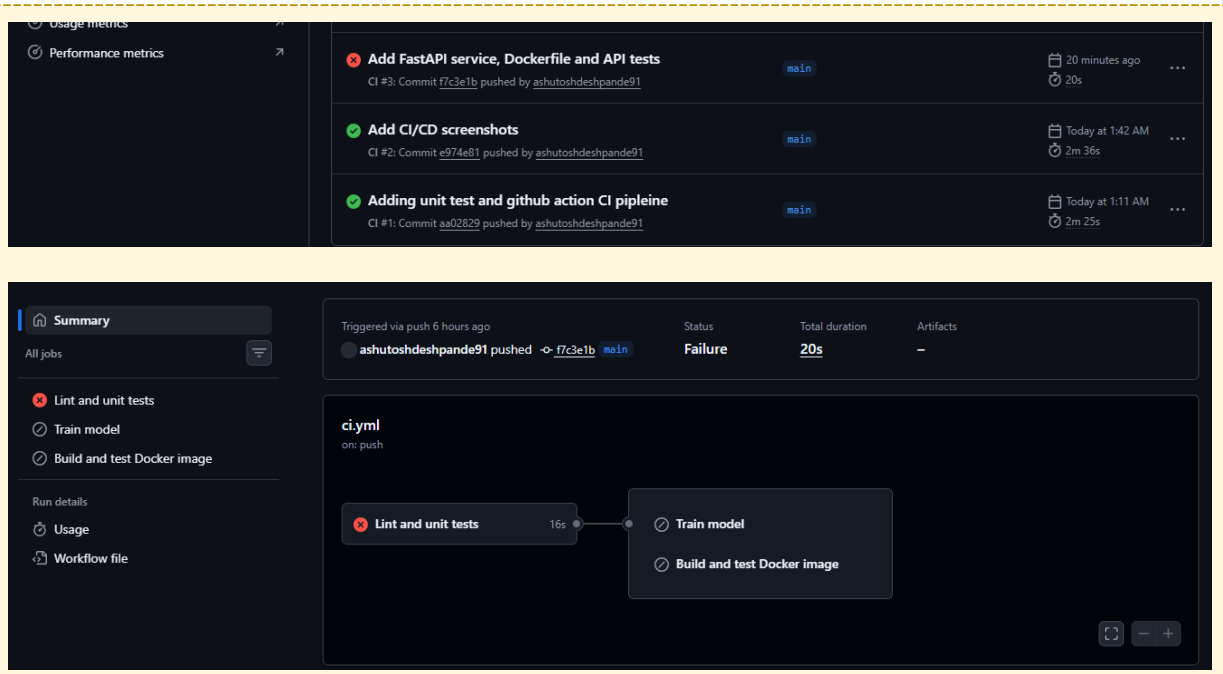


Figure 6: The container answering a prediction request inside CI.

7.2 A failing run

During development the pipeline failed once, which shows that it does what it should. After I added the API, the install step stopped with `ResolutionImpossible` because two requirements files pinned pandas to different versions (3.0.5 and 3.0.6). The log named both versions, so the cause was easy to find. I fixed it by taking the shared package versions in `requirements-api.txt` directly from `requirements.txt`, so they cannot disagree. The next run passed.



Figures 7: A failed run

8. Containerisation and the prediction API

8.1 API

The service is built with FastAPI. `POST /predict` accepts the 13 patient values as JSON and returns the prediction, the confidence in it and the probability of disease. Inputs are validated with Pydantic, with plausible ranges for each field, so malformed requests get HTTP 422 with a message naming the problem instead of reaching the model. `GET /health` reports whether the model loaded and returns 503 if it did not. `/docs` serves interactive documentation.

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
       "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'

{"prediction": 1, "confidence": 0.85, "probability_disease": 0.85, "label": "heart
disease"}
```

8.2 Dockerfile

- **Slim base image** (`python:3.13-slim`) and an install of only `requirements-api.txt`, so the image does not carry MLflow, plotting or test libraries.
- **Layer ordering:** requirements are copied and installed before the source code, so editing code does not repeat the slow dependency install.
- **Model included** in the image (`models/heart_model.joblib`), so the container is self-contained.
- **Non-root user** and a `HEALTHCHECK` that calls `/health`.
- A `.dockerignore` keeps data, notebooks, screenshots, tests and the virtual environment out of the build context.

The assignment requires proof that the model serves correctly in an isolated environment. The CI job (Figure 6) builds the image from a clean checkout and tests the running container on every push; Figures 8 and 9 show the same locally.

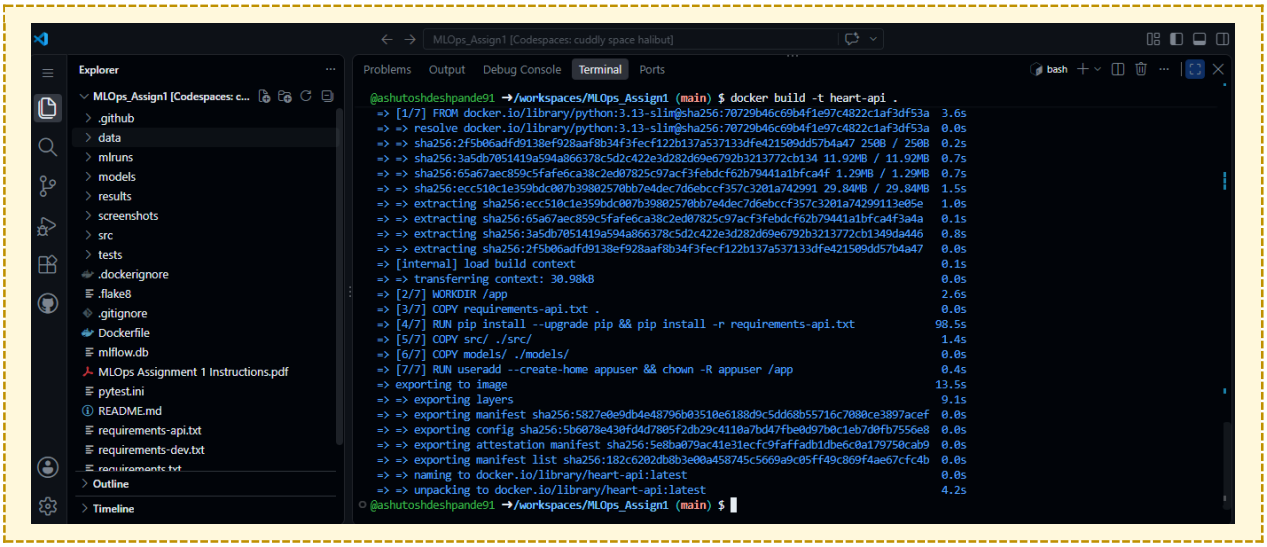


Figure 8: Building and running the container.

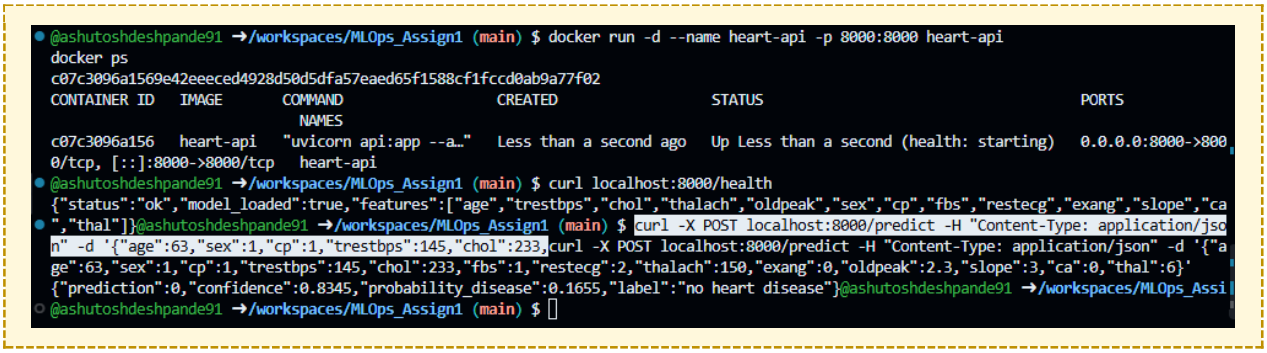


Figure 9: Health check and prediction from the container.

9. Kubernetes deployment

The container is deployed to a local Minikube cluster (Docker driver). My laptop could not run Docker Desktop, so I ran Docker and Minikube inside a GitHub Codespace. The manifests are in `k8s/`.

9.1 Manifests

- **Deployment** (`k8s/deployment.yaml`): 2 replicas of the `heart-api` image with a rolling update strategy (`maxUnavailable: 0`, `maxSurge: 1`), so a new version is started before an old pod is removed. A **readiness probe** on `/health` sends traffic to a pod only after the model has loaded. A **liveness probe** restarts a pod that stops responding. CPU and memory requests and limits keep one pod from starving the node.
- **Service** (`k8s/service.yaml`): type `LoadBalancer`, port 80 forwarding to container port 8000, selecting the pods by label.

The image is built locally and loaded into the cluster with `minikube image load`, so the Deployment uses `imagePullPolicy: Never`.

9.2 Deploy and verify

```
minikube start --driver=docker
docker build -t heart-api:latest .
minikube image load heart-api:latest
kubectl apply -f k8s/
kubectl get pods,deployments,services
kubectl port-forward service/heart-api 8080:80
curl http://localhost:8080/health
```

I reached the service with `kubectl port-forward`; the Service itself is a LoadBalancer, which on a real cloud cluster, or on Minikube with `minikube tunnel`, receives an external IP. The assignment allows a local cluster, so there is no public URL. Access instructions are in the repository README.

```
@ashutoshdeshpande91 ➜/workspaces/MLOps_Assign1 (main) $ kubectl apply -f k8s/
kubectl get pods
kubectl get deployments
kubectl get services
deployment.apps/heart-api created
service/heart-api created
NAME                         READY   STATUS              RESTARTS   AGE
heart-api-768d8fcdbd-6rzsd   0/1     ContainerCreating   0          0s
heart-api-768d8fcdbd-hxchv   0/1     ContainerCreating   0          0s
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
heart-api   0/2     2            0           1s
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
heart-api    LoadBalancer   10.107.105.153   <pending>     80:30754/TCP   1s
kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP        19m
@ashutoshdeshpande91 ➜/workspaces/MLOps_Assign1 (main) $ kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
heart-api-768d8fcdbd-6rzsd   1/1     Running   0          48s
heart-api-768d8fcdbd-hxchv   1/1     Running   0          48s
@ashutoshdeshpande91 ➜/workspaces/MLOps_Assign1 (main) $ kubectl get services
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP      PORT(S)        AGE
heart-api    LoadBalancer   10.107.105.153   10.107.105.153   80:30754/TCP   2m51s
kubernetes   ClusterIP      10.96.0.1        <none>           443/TCP        22m
```

Figure 10: The deployment running on Kubernetes.



Figures 11: Requests served by the pods.

10. Monitoring and logging

10.1 Request logging

The API logs each request once, with a request ID, the method, path, status code and latency in milliseconds, and logs each prediction with its class and confidence. The request ID is also returned in an `X-Request-ID` response header, so a client report can be matched to a log line. In Docker and Kubernetes the logs are available with `docker logs` and `kubectl logs`.

10.2 Metrics

`GET /metrics` exposes Prometheus metrics. The path label uses the route template, and unknown URLs share one label, so a scanner hitting random URLs cannot create unlimited time series.

Metric	Type	What it shows
`api_requests_total`	counter	requests by method, path and status code
`api_request_duration_seconds`	histogram	latency per endpoint (used for average and p95)
`api_predictions_total`	counter	predictions by class (disease or no disease)
`api_prediction_confidence`	histogram	confidence of the predicted class

10.3 Prometheus and Grafana

`monitoring/docker-compose.yml` starts the API, Prometheus (scraping every 5 seconds) and Grafana. Grafana is provisioned automatically with a Prometheus data source and a dashboard of 9 panels: total requests, total predictions, error rate, average latency, requests per second by endpoint, p95 latency, responses by status code, predictions by class per minute and median prediction confidence. I sent a mix of valid and deliberately invalid requests to populate the dashboard, including the 422 errors.

In GitHub Codespaces, traffic between containers was blocked, so Prometheus and Grafana reach the other services through the host's published ports (`host.docker.internal`). This setup also works on an ordinary Docker installation.



Figure 12: Prometheus scraping the API.

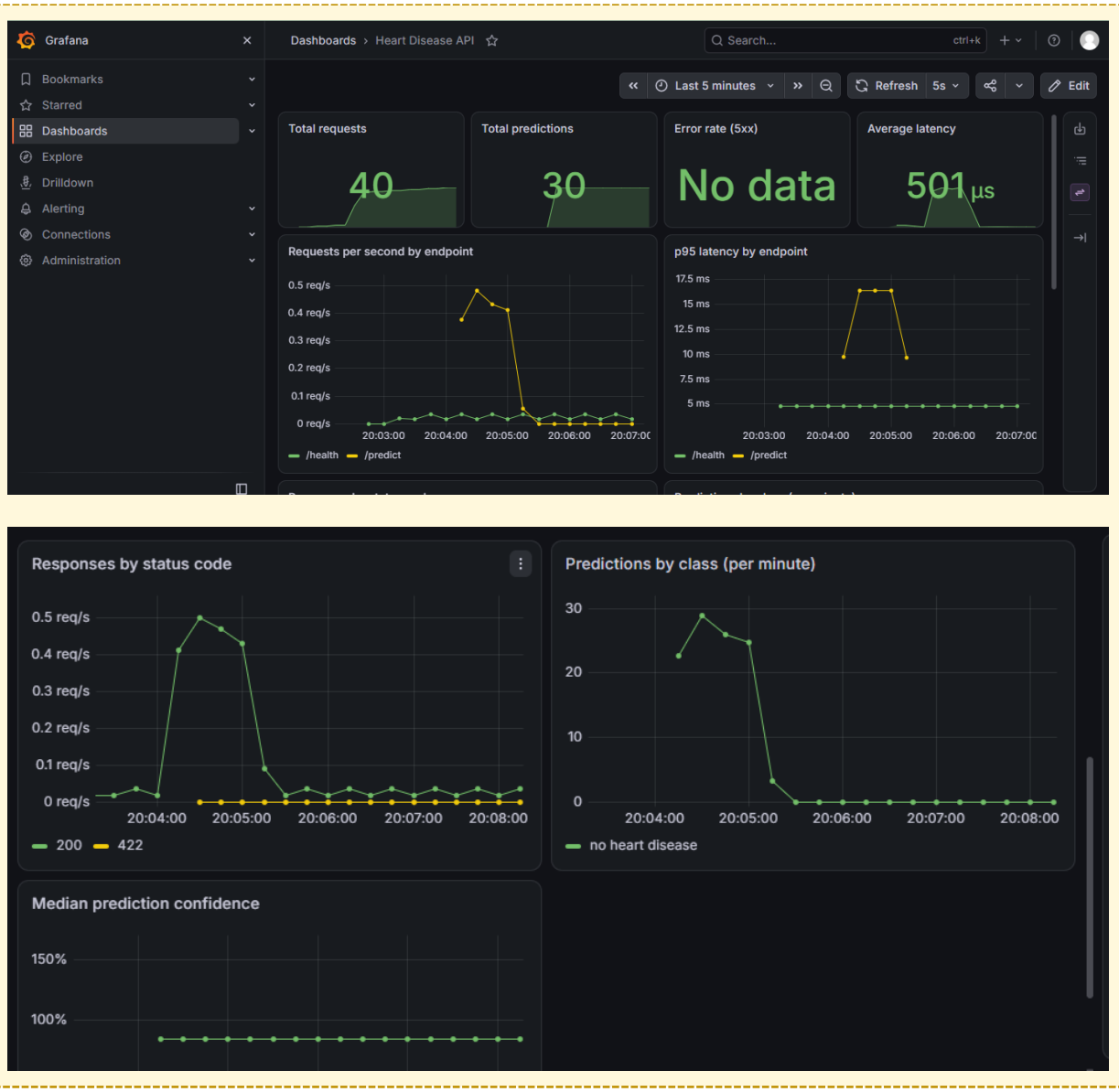


Figure 13: The Grafana dashboard.

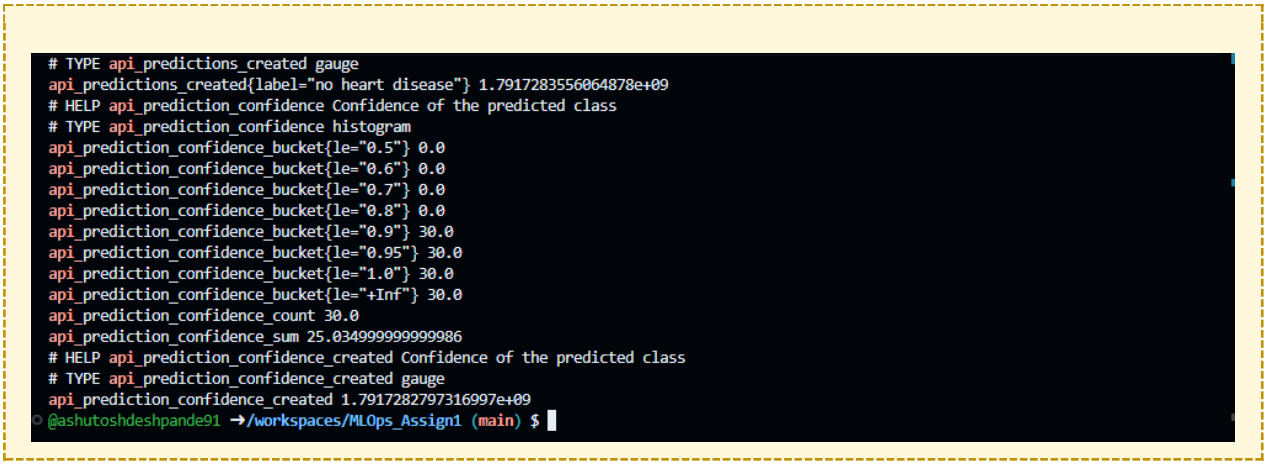


Figure 14: Request logging.

11. Problems met and how I solved them

- **Model serialisation in MLflow.** The newest MLflow saves scikit-learn models in a safe format that refuses tree models such as Random Forest. I log the model with `cloudpickle` instead, which is acceptable because I trained and saved the file myself.
- **Conflicting dependency pins.** Described in Section 7.2. Deriving the API requirements from the training requirements removed the possibility of a mismatch, and it also guarantees the container uses the same scikit-learn version as the one that trained the model.
- **No local Docker.** Docker Desktop could not start on my laptop (virtualisation was not available to it), so I used GitHub Codespaces for Docker, Minikube and the monitoring stack. Files were kept in sync through Git.
- **Minikube networking.** The cluster node stayed `NotReady` because its default network add-on could not pull its image; restarting with the `bridge` network plugin fixed it.
- **Blocked container-to-container traffic.** Prometheus could not reach the API by service name. Testing from inside the container showed that the host's published port worked, so I routed monitoring through `host.docker.internal`.

12. Limitations and future work

- **Small dataset.** 303 patients from one source, so the results have wide uncertainty and may not carry over to other populations. This is a learning project and not a medical device.
- **Threshold.** The 0.5 decision threshold is not tuned. For screening, a lower threshold that raises recall would be worth evaluating.
- **Model delivery.** The model file is baked into the image. A model registry (for example the MLflow Model Registry) would allow versioned models and rollbacks without rebuilding the image.
- **Security.** The API has no authentication or rate limiting, which a real deployment would need.
- **Monitoring.** The dashboard tracks service health and prediction mix, not data drift or model quality. Drift detection and alert rules in Prometheus would be natural additions.
- **Deployment.** A single-node local cluster, with kubectl manifests. Ingress, a Helm chart and a cloud cluster with automatic deployment from CI would complete the delivery path.

13. Repository and access instructions

Repository: https://github.com/ashutoshdeshpande91/MLOps_Assign1

Deployed API: there is no public URL; the service runs locally. To reach it:

- **Docker:** `docker build -t heart-api .`, then `docker run -p 8000:8000 heart-api`, then open `http://localhost:8000/docs`.
- **Kubernetes:** follow Section 9.2, then use `http://localhost:8080` through the port-forward.
- **With monitoring:** `docker compose -f monitoring/docker-compose.yml up -d --build`. Prometheus is at port 9090 and Grafana at port 3000 (login admin / admin).

The `README.md` in the repository lists every command in order.

References

- Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R. Heart Disease dataset. UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/45/heart+disease
- Documentation of scikit-learn, MLflow, FastAPI, Prometheus, Grafana, Docker and Kubernetes (official sites).